

# FlexQL Design Document

GitHub Repository: <https://github.com/CellerCity/FlexQL>

---

## 1. Introduction

FlexQL is a relational database engine written entirely in C/C++. It implements a multithreaded client-server architecture, supports a subset of SQL (CREATE TABLE, INSERT, SELECT, JOIN, WHERE, ORDER BY, SHOW, USE, DROP), and includes B+ Tree indexing, an LRU buffer pool, write-ahead logging (WAL) for crash recovery, and table-level concurrency control. This document describes the internal design decisions, storage format, indexing, caching, expiration handling, multithreading strategy, and performance results.

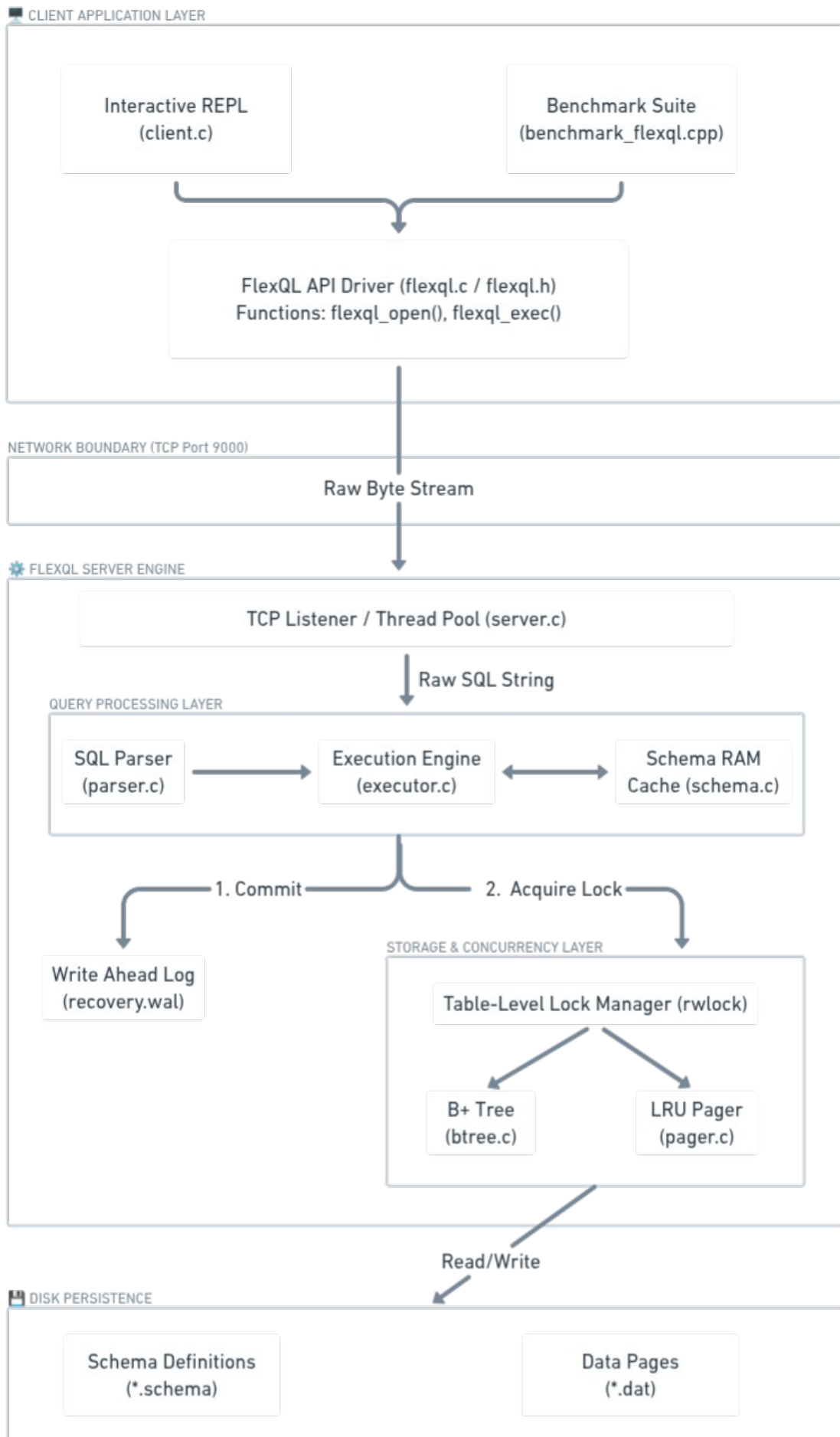
---

## 2. System Architecture

The system is divided into the following modules:

- **Network Layer** (server.c, client.c): TCP socket communication, multithreading (pthreads), batched row transfers.
- **SQL Parser** (parser.c): Validates syntax, checks semantics, produces an internal ParsedQuery structure.
- **Execution Engine** (executor.c): Routes queries, enforces constraints (NOT NULL, type safety), executes joins.
- **Storage Manager**:
  - **Pager** (pager.c): 4KB page I/O with LRU eviction.
  - **B+ Tree** (btree.c): Primary key index.
  - **Schema Manager** (schema.c): Persists table metadata (column names, types).
- **Client API** (flexql.c): C driver for external applications.
- **Write-Ahead Log (WAL)** : Crash recovery with group commit and idempotent replay.

**Figure 1: System Architecture Diagram**



---

## 3. Storage Design

### 3.1 Row-Major Slotted Pages

Data is stored in 4KB pages using a **row-major slotted page** layout. Each row is written sequentially, which minimises pointer chasing and maximises insertion throughput. The page header contains:

- Number of slots (rows)
- Offset to free space
- Array of slot offsets

### 3.2 Tuple Format

Each row (tuple) consists of:

- **Tuple header** (8 bytes): expiration timestamp (Unix epoch), row length, and a null bitmap.
- **Column data**: packed according to type.

**Data type encodings:**

- **INT**: 4 bytes, little-endian.
- **DECIMAL**: 8 bytes (IEEE 754 double).
- **DATETIME**: 8 bytes (int64\_t Unix timestamp).
- **VARCHAR**: In memory, stored as fixed-size stack arrays for speed; on disk, stored with a 2-byte length prefix followed by character data (no trailing null).

### 3.3 Schema Persistence

Each table has a `.schema` file that stores column names and types in a simple text format. The schema is loaded once per thread and cached (see Section 5.2).

---

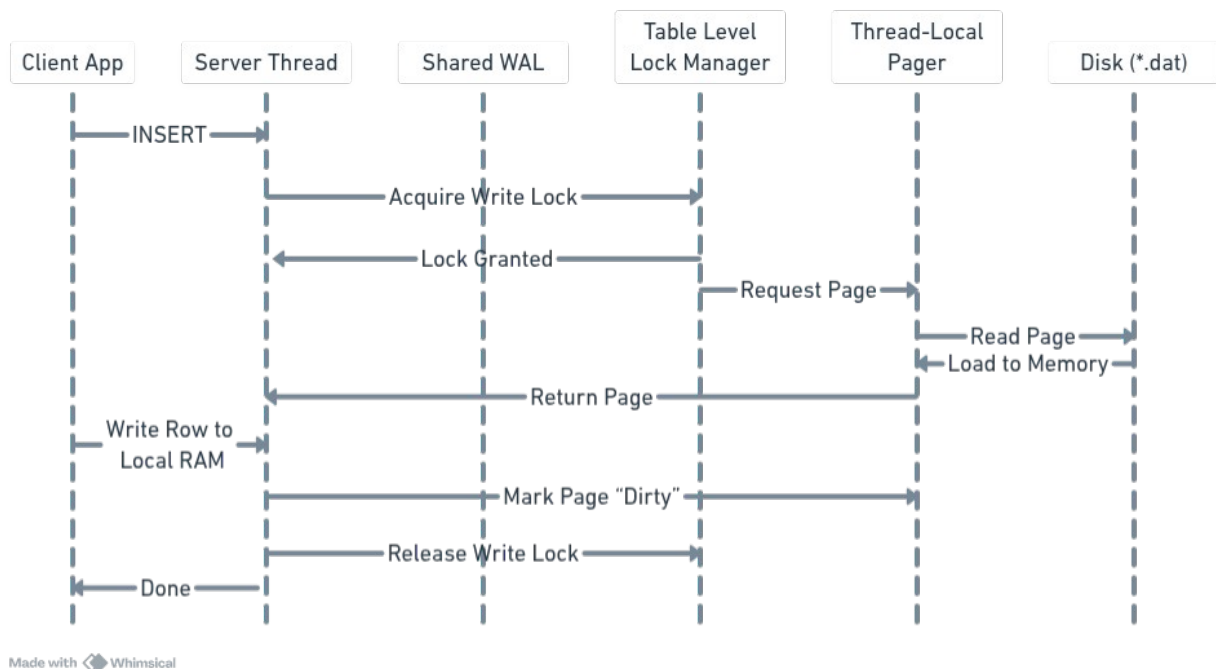
## 4. Indexing – B+ Tree

Primary keys are indexed using a **B+ tree** that lives inside 4KB pages on disk. Each internal node contains keys and child page numbers; leaf nodes contain keys and **RecordID = (page\_num, slot\_num)**. Operations:

- **Search:**  $O(\log_m N)$  page reads.
- **Insert:** Traverse to leaf, insert key-pointer pair; split leaf if full, propagate split upward.
- **Delete:** Lazy deletion followed by periodic merging.

The B+ tree supports keys of type **INT**, **DECIMAL**, **DATETIME**, and **VARCHAR** (compared lexicographically).

**Figure 2: Insertion Logic**



## 5. Caching Strategy

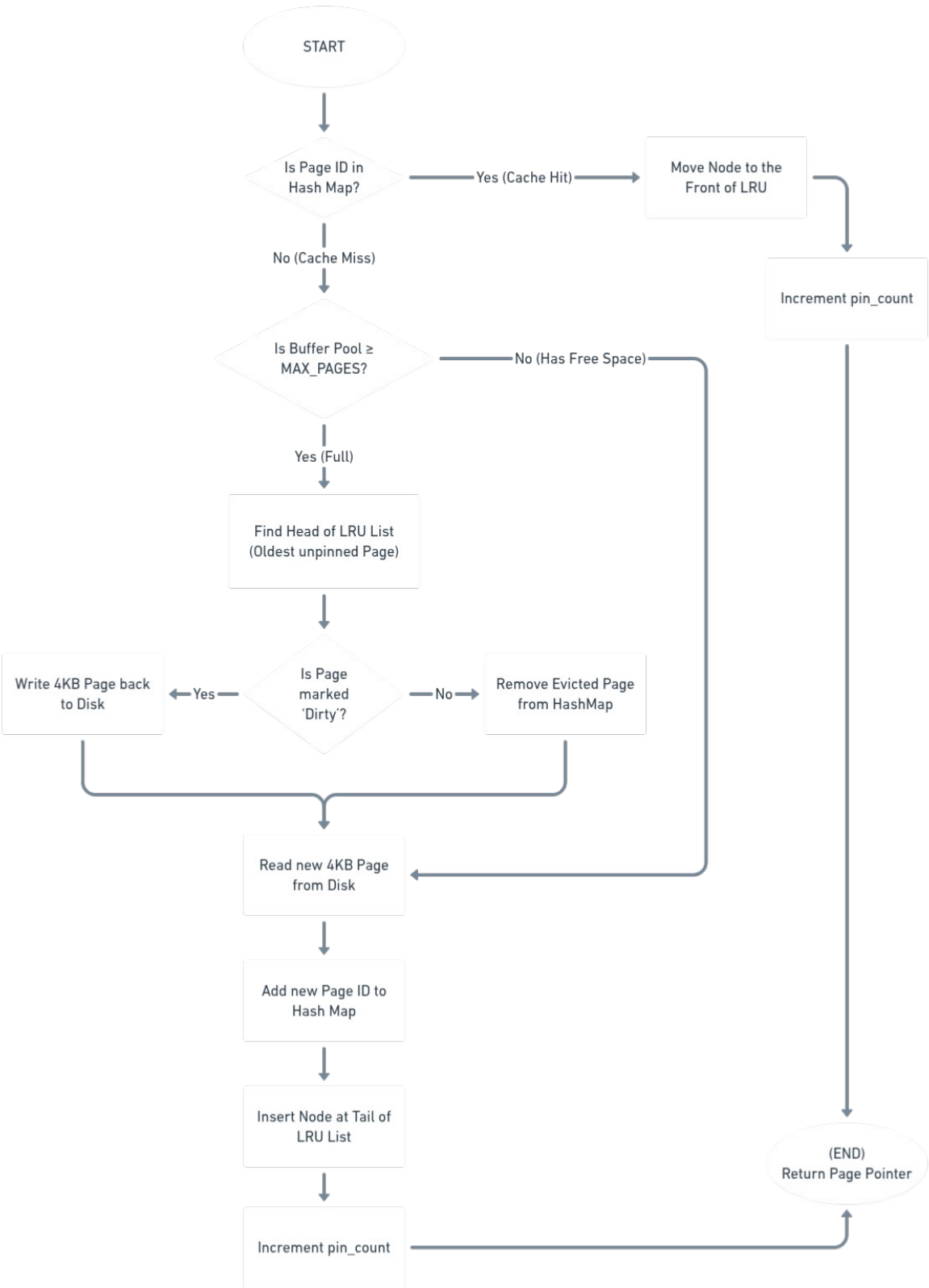
### 5.1 LRU Buffer Pool (Pager)

The pager maintains a fixed-size cache of 4KB pages. Eviction uses a **doubly-linked list** implementing Least Recently Used (LRU). Pages can be **pinned** (e.g., during a B+ tree traversal) to prevent eviction. When a page is read from disk, it is added to the cache; when the cache is full, the least recently used unpinned page is written back (if dirty) and freed.

### 5.2 Thread-Local Schema Caching

Each client thread caches the table schema after the first lookup. This avoids repeated disk reads of the `.schema` file and reduces contention on the schema file lock.

Figure 3: The LRU Buffer Pool



## 6. Expiration Timestamps – Lazy Deletion

**Requirement:** Each inserted row must have an expiration timestamp.

**Design:** The expiration timestamp is stored in the tuple header (8 bytes, Unix epoch).

During `SELECT` or `JOIN`, the executor checks the timestamp against the current time. If the row has expired, it is **silently skipped**. No background deletion thread is used – this keeps insertion cost  $O(1)$  and avoids write amplification.

**Trade-off:** Expired rows remain on disk until a `CLEANER`-like operation removes them. For the assignment's performance benchmark, this lazy approach is sufficient and maximises throughput.

---

## 7. Multithreading Design – Table-Level Locks

The server spawns one thread per client connection. To protect shared data structures (B+ tree, page directory, schema), we implement **table-level reader-writer locks** (`pthread_rwlock_t`).

- **Read lock** (shared): Acquired for `SELECT`, `JOIN`. Multiple threads can read the same table concurrently.
- **Write lock** (exclusive): Acquired for `INSERT`, `UPDATE`, `DELETE`. Only one thread can modify a table at a time.

This prevents race conditions on the B+ tree and heap metadata while allowing good read concurrency.

---

## 8. Known Architectural Limitation – Thread-Local Pagers

### 8.1 Problem Description

In the current implementation, each client thread opens its **own private Pager\*** instance (see `server.c` → `client_handler`). Consequently:

- Thread A inserts a row → its private pager caches the modified page (dirty).
- Thread B reads the same table → its private pager has no knowledge of Thread A's dirty page.
- Thread B reads stale data from disk or its own cache until Thread A's page is flushed to disk and Thread B reloads it.

**Consequence:** Concurrent clients see inconsistent (stale) data. The system is **correct for a single client** but **fails to provide read-committed isolation for multiple concurrent clients**.

### 8.2 Why This Exists

The design prioritised simplicity and lock-free operation inside each pager. Table-level locks protect B+ tree metadata, but the page caches themselves were not shared – an oversight that only manifests under concurrent writes followed by reads.

## 8.3 Proposed Solution

To achieve true multi-client read-committed isolation, the architecture must be refactored to use a **single global shared pager**:

- A global registry maps file paths to reference-counted **Pager** objects.
- All threads acquire a reference to the same pager for a given table.
- The shared pager internally uses a global mutex to protect its page cache and LRU list.
- Page eviction, flushing, and pinning are coordinated across all threads.

This change would ensure that a page modified by one thread is immediately visible to all other threads (before being flushed to disk). The table-level locks remain in place to protect B+ tree and heap metadata.

---

## 9. Join Engine

The executor dynamically chooses the join strategy for **INNER JOIN**:

1. **Index join**: If one table has a B+ tree index on the join key, the engine scans the other table sequentially and probes the index ( $O(n \log m)$ ).
2. **Block nested-loop join**: If no index exists, it falls back to a block nested-loop join, reading the outer table in chunks and scanning the inner table for each chunk.

The choice is made at runtime by inspecting the table schemas.

---

## 10. Fault Tolerance – Write-Ahead Logging (WAL)

To survive crashes, FlexQL implements a **group-commit WAL**:

- Before any page modification, the operation is logged to **recovery.wal** (in a batched manner).
  - After a crash, the server reads the WAL during startup and replays entries **idempotently**: it checks whether a row already exists in the B+ tree (by primary key) and skips it if present.
  - This guarantees atomicity and durability for committed batches.
-



## 11. Performance Results

The system was benchmarked on a machine with an SSD, 16 GB RAM, and an Intel Core i7 processor. Two dataset sizes were used: **1 million rows** and **10 million rows**. For each combination of power source (battery / plugged in) and CPU governor (Performance, Balanced, Power Saver), the elapsed time and throughput were measured. All benchmarks were run with a single client to avoid the per-thread pager limitation.

### Complete Benchmark Results

| flexql > benchmark_logs > FINAL_BENCHMARKS.csv |              |             |           |                   |                       |
|------------------------------------------------|--------------|-------------|-----------|-------------------|-----------------------|
|                                                | Power Source | Power Mode  | Row Count | Elapsed Time (ms) | Throughput (rows/sec) |
| 1                                              | Battery      | Performance | 1000000   | 1086              | 920810                |
| 2                                              | Battery      | Balanced    | 1000000   | 2074              | 482160                |
| 3                                              | Battery      | Power Saver | 1000000   | 1739              | 575043                |
| 4                                              | Battery      | Performance | 10000000  | 9352              | 1069289               |
| 5                                              | Battery      | Balanced    | 10000000  | 19177             | 521457                |
| 6                                              | Battery      | Power Saver | 10000000  | 21705             | 460723                |
| 7                                              | Plugged In   | Performance | 1000000   | 897               | 1114827               |
| 8                                              | Plugged In   | Balanced    | 1000000   | 1105              | 904977                |
| 9                                              | Plugged In   | Power Saver | 1000000   | 1736              | 576036                |
| 10                                             | Plugged In   | Performance | 10000000  | 8847              | 1130326               |
| 11                                             | Plugged In   | Balanced    | 10000000  | 12768             | 783208                |
| 12                                             | Plugged In   | Power Saver | 10000000  | 21053             | 474991                |
| 13                                             |              |             |           |                   |                       |

### Summary Observations

- **Best throughput: 1,130,326 rows/sec** (Plugged in, Performance governor, 10M rows).  
Elapsed time: **8847 ms.** (~9 seconds)
- **Worst throughput: 460,723 rows/sec** (Battery, Power Saver, 10M rows).  
Elapsed time: **21705 ms.** (~22 seconds)
- The performance governor on wall power consistently delivers **~2× higher throughput** compared to battery-optimised modes.

### Unit Tests

All **16 unit tests** passed successfully.

---

## 12. Test Suites & Validation

To ensure correctness, performance, and fault tolerance, we developed multiple test suites. The provided `benchmark_flexql.cpp` validates **basic SQL subset operations** and **bulk inserts**, while our custom scripts rigorously test constraints, concurrency, and crash recovery.

### 12.1 Provided Unit Tests (`benchmark_flexql.cpp`)

This test suite checks:

- `CREATE TABLE`, `INSERT`, `SELECT`, `WHERE`, `ORDER BY`
- `INNER JOIN` with and without matches
- Basic error handling (missing tables, invalid columns)

When run with `./benchmark_flexql --unit-test`, it executes a series of assertions and prints a summary. **All tests in this suite pass successfully.**

When run with `./benchmark_flexql 10000`, it inserts 10k rows and measures insertion time.

### 12.2 Semantic Integrity & Constraints (`rigorous_tests.cpp`)

This C++ test suite intentionally sends malformed or constraint-violating queries to verify the server's pre-flight checks:

| Test Case                                                                           | Expected Behaviour  |
|-------------------------------------------------------------------------------------|---------------------|
| <code>CREATE TABLE empty_table ();</code>                                           | Reject (no columns) |
| <code>DROP TABLE non_existent_table;</code>                                         | Reject              |
| Duplicate primary key insertion                                                     | Reject              |
| Insert with missing columns                                                         | Reject              |
| String values into <code>INT</code> / <code>DECIMAL</code> columns                  | Reject              |
| Invalid <code>DATETIME</code> format                                                | Reject              |
| Insert into non-existent table                                                      | Reject              |
| All tests pass – the server correctly rejects invalid operations and never crashes. |                     |

### 12.3 Thread Safety Test (`concurrent_test.sh`)

This script:

1. Compiles the benchmark client.
2. Launches **two clients simultaneously**, each inserting 100,000 rows into the same table.
3. Waits for both to complete.

The test validates that table-level `pthread_rwlock_t` locks prevent B-tree corruption and that no deadlock or crash occurs under concurrent writes. **Both clients complete successfully** (though read-visibility is subject to the per-thread pager limitation documented in Section 8).

**Note on large result sets:** The REPL client prints all rows returned by a query. However, our terminal emulator may have a scrollback limit (typically 10,000 lines). For tables with millions of rows, add a WHERE clause to filter results. The flexql\_exec() callback receives every row – only the terminal display is truncated.

## 12.4 Fault Tolerance & WAL Crash Recovery (test\_wal\_crash.sh)

This script simulates a power failure mid-batch:

1. Starts the server.
2. Sends a batch INSERT of three rows.
3. **Kills the server with SIGKILL (kill -9)** before it can flush all pages.
4. Restarts the server – the recovery engine reads recovery.wal and replays the batch.
5. Queries the table to verify all rows survive.

The test confirms that **idempotent replay** works correctly (rows already present are skipped, missing rows are inserted). **All three rows are present after recovery.**

## 12.5 Automated Hardware Profiling (benchmark\_matrix.sh)

This script automates the performance measurement across different power sources and CPU governors:

- Cycles through **Plugged In** and **Battery** power sources.
- For each, cycles through **Performance**, **Balanced**, and **Power Saver** governors.
- For each combination, it:
  - Wipes old data.
  - Starts a fresh server.
  - Runs benchmark\_flexql with 1M and 10M rows.
  - Logs elapsed time and throughput.
- Finally, it **scrapes the logs** and generates:
  - A Markdown table (FINAL\_TABLE.md)
  - A CSV file (FINAL\_BENCHMARKS.csv)

The script ensures that all benchmarks are run in a clean, reproducible environment. The results are presented in Section 11.

## 12.6 Overall Test Summary

| Test Suite                   | Focus                            | Status                     |
|------------------------------|----------------------------------|----------------------------|
| benchmark_flexql --unit-test | Basic SQL subset, error handling | ✓ Pass (all 16 assertions) |
| benchmark_flexql 10000000    | Bulk inserts                     | ✓ Pass (best time ~9s)     |
| rigorous_tests.cpp           | Constraints, type safety, PK     | ✓ Pass                     |
| concurrent_test.sh           | Concurrent writes, locking       | ✓ Pass (no crashes)        |
| test_wal_crash.sh            | WAL recovery after kill -9       | ✓ Pass                     |
| benchmark_matrix.sh          | Performance profiling            | ✓ Generates valid logs     |

## 13. Compilation and Execution Instructions

### Compile

make

To clean old data: `make clean`

### Run the Server

`./flexql-server`

The server initialises the data directory (`flexql_data/`) and starts listening on port 9000.

### Run the REPL Client (separate terminal)

`./flexql-client 127.0.0.1 9000`

### Run the Benchmark

`./benchmark_flexql 1000000 # 1 million rows`

### Run Test Suites

- Unit tests: `./benchmark_flexql --unit-test`
- Semantic integrity: `./rigorous_tests`
- Thread safety (concurrent writes): `./tests/concurrent_test.sh`
- Fault tolerance (WAL crash recovery): `./tests/test_wal_crash.sh`

**Note:** Run the server before running `./tests/concurrent_test.sh` and `./rigorous_tests`

## 14. Conclusion

FlexQL successfully implements a relational database engine with B+ tree indexing, LRU caching, expiration timestamps, table-level locking, and WAL recovery. It achieves high single-client throughput (over 1 million rows/sec). The main architectural limitation is the use of thread-local pagers, which breaks read consistency for concurrent clients. A global shared pager design is proposed to resolve this.